

Real-Time Adaptive Music Recommendation

Project Proposal (Max 2 Pages)

Team: **Spoty Boys** — Open Source Base: **Navidrome** (19.6k stars)

OSS: <https://github.com/navidrome/navidrome>

Teammates: **Vishal Jha, Yonghyeon Shin, Dawei Sun** — Deployment: **Chameleon**

Date: March 3, 2026

1. Public-Facing Service (Joint, 0.5 pt) 1.1 What service are we building (not the ML feature)? Self-Hosted Music Streaming Service is a **personal music server** where users browse a music library, stream tracks to a web/mobile player, and manage playlists (realized using Navidrome). The ML feature is complementary: it recommends what to play next.

1.2 Audience + context + scale assumptions

- **Primary audience:** students/hobbyists who self-host a personal library; small groups sharing a music server.
- **Context:** streaming sessions on web/mobile; recommendations are requested mainly at **song end** or **skip**.
- **Design scale:** **100 concurrent listeners** for load tests (typical demo < 20).

1.3 Open-source base and why it is suitable We build on **Navidrome** because:

- Mature OSS with a realistic playback loop to generate user interaction data.
- Clear entity IDs + metadata pipeline, enabling event/impression logging and offline training.
- Feasible end-to-end hosting on Chameleon; the recommender integrates as a complementary service via API/plugin hooks.

2. Complementary ML Feature Design (Joint, 2.0 pt) 2.1 Problem statement + intended

outcome Problem: In a self-hosted music library, users often rely on manual search or static playlists; the experience lacks adaptive personalization (especially “what to play next”).

Outcome: After each end/skip event, the system returns **Top-K next tracks** and updates the user’s queue, adapting to both **short-term mood** (session) and **long-term taste** (profile).

2.2 Inputs → Model → Outputs (high-level)

- **Inputs:** recent session songs (last 1–5), user history aggregates, candidate song metadata/embeddings.
- **Model outputs:** Top-K recommended **song IDs** (optionally with scores + model version).
- **Action in system:** write recommended IDs into the **Up Next queue** (target length 5) with constraints (no repeats, filter dislikes, basic diversity).

2.3 Feedback loop (strict requirement) + retraining usage Feedback captured in production (plugin + client logging):

- **Implicit:** play/start, listen duration (% complete), skip (early vs late), repeat, manual search+play, queue remove/add.
- **Explicit:** like/dislike (optional: “more like this” / “not in mood”).

How feedback is used for retraining:

- **Labeling rules:** positive = like OR repeat OR completion >80%; negative = dislike OR early skip <20–30s; neutral = partial listen.
- **Training examples:** (session/user features, candidate features, impression list, outcome) → ranking dataset.
- **Retraining:** periodic retrain (nightly/weekly at project scale) + triggered retrain if KPI drift (skip-rate) or fallback spikes.

2.4 Design spec summary (for TA review)

3. External Dataset(s) and Lineage (Joint, 0.5 pt) 3.1 Dataset list + why aligned

- **Primary dataset:** **The MTG-Jamendo Dataset** [MTG-Jamendo](#).
Lineage: curated/published for music information retrieval research; provides real track-level metadata/features at scale.

Alignment: provides **content/metadata features** for candidate generation + cold-start similarity; user feedback comes from our deployed system.

4. Training Plan (Training Member, 2.0 pt)

- **Two-stage recommender:**
 - **Retrieval:** candidate generation via audio embeddings extracted using pretrained PANNs. Top- N nearest neighbors are retrieved using vector similarity to ensure semantic consistency and low latency.
 - **Ranking:** lightweight MLP ranker over (context, session, candidate) features, outputting engagement probability.
- **Retraining plan:**
 - Periodic retraining on newly collected impressions and outcomes.
 - Triggered retraining if performance drift (e.g., skip-rate increase) is detected.
 - Publish versioned model artifacts to support controlled deployment and rollback.

5. Serving Plan (Serving Member, 2.0 pt) 5.1 Operational requirements with justification (1 pt)

Serving requirement (summary)	Decision / Target + justification
Inference call pattern	Refresh on end/skip; maintain Up-Next queue of 5. End/skip is when the next rec matters; queue buffer avoids playback gaps and avoids calling the model on every UI interaction.
Throughput (RPS)	Steady: 0.5–1 RPS; Burst: 5 RPS. From 100 active users and ~ 1 trigger/3 min $\Rightarrow$ RPS $\approx 100 \times 0.33/60 \approx 0.48$; burst covers skip storms + synchronized refresh.
Latency SLO + timeout	p50 < 100ms; p95 < 300ms; timeout 800–1000ms + fallback. Fast queue refill feels instant; hard timeout prevents stalled playback; fallback guarantees continuity.
Output contract	Return Top-5 song IDs (optional scores + model version). IDs are the native action unit (queue/play); scores help debugging and evaluation.
Candidate strategy + caching	Two-stage: ~ 100 candidates $\rightarrow$ rank $\rightarrow$ Top-5; cache 30–120s (Redis session). Avoid scoring full catalog; caching reduces repeated scoring during bursts while short TTL keeps adaptivity.
Fallbacks + storage + observability	Fallback: popularity / content-similarity; Storage: Postgres events, Redis session, object store artifacts; Metrics: latency, RPS, errors, fallback-rate, cache hit-rate, skip & completion. Baselines keep playback safe and measurable; storage ensures retraining + reproducibility; metrics validate both system health and ML impact.

5.2 Model output $\rightarrow$ system outcome (1 pt)

- **Output contract:** JSON containing `recommended_song_ids` (Top-5) + optional scores + `model.version`.
- **System action:** update **Up Next queue** after end/skip; cache results for 30–120s; debounce calls (max 1/sec/user).
- **Constraints:** filter dislikes; no repeats in last M (e.g., 20); limit consecutive same-artist; small exploration ϵ .

6. Data Flow and Data Quality Plan (Data Member, 2.0 pt) 6.1 Data flow: real-time inference vs training (1 pt)

- **Real-time inference data:** end/skip triggers a request with `user_id`, session context, and recent song IDs; service reads session/cache state and embeddings; returns Top-5.
- **Training data:** plugin + client logging writes **events** (plays/skips/duration/likes) and **impressions** (what was shown/recommended) to Postgres; batch ETL produces training tables.
- **Feedback capture:** all actions are timestamped and include model version to enable offline regression and drift analysis.

6.2 Candidate selection + leakage avoidance (1 pt)

- **Candidate selection:** retrieval returns 50–200 nearest neighbors; mix in popularity and exploration; negatives sampled from shown-but-not-chosen impressions.
- **Split strategy:** time-based split (train on past, validate/test on future).
- **Leakage avoidance:** compute features only from data available at decision time; avoid label-derived features in inference; ensure no future events enter training rows.